3.9 Batch normalization

- training deep neural networks is difficult, and getting them to converge in a reasonable amount of time can be tricky
- in this section, we describe *batch normalization*, a popular and effective technique that consistently accelerates the convergence of deep networks
- together with residual blocks – covered later, in Section 3.10 – batch normalization has made it possible to routinely train networks with over 100 layers
- to motivate batch normalization, let us review a few practical challenges that arise when training machine learning models, and neural networks in particular

3.9 Batch normalization

- first, choices regarding data preprocessing often make an enormous difference in the final results
- our first step when working with data is to standardize our input features to each have a mean of zero and variance of one
- intuitively, this standardization plays nicely with our optimizers, because it puts the parameters *a priori* at a similar scale
- second, for a typical MLP or CNN, as we train, the variables (e.g., affine transformation outputs in MLP) in intermediate layers may take values with widely varying magnitudes: both along the layers from the input to the output, across units in the same layer, and over time due to our updates to the model parameters

3.9 Batch normalization

- the inventors of batch normalization postulated informally that this drift in the distribution of such variables could prevent the convergence of the network
- intuitively, we might conjecture that, if one layer has variable values that are 100 times that of another layer, this might need compensatory adjustments in the learning rates
- third, deeper networks are complex and easily capable of overfitting; this means that regularization becomes more critical

3.9 Batch normalization

- batch normalization is applied to individual layers (optionally, to all of them) and works as follows: in each training iteration, we first normalize the inputs (of batch normalization) by subtracting their mean and dividing by their standard deviation, where both are estimated based on the statistics of the current mini-batch
- next, we apply a scale coefficient and a scale offset
- it is precisely due to this *normalization* based on *batch* statistics that *batch normalization* derives its name

3.9 Batch normalization

- note that, if we tried to apply batch normalization with mini-batches of size 1, we would not be able to learn anything
- that is because, after subtracting the means, each hidden unit would take value 0
- as we might guess, since we are devoting a whole section to batch normalization, with large enough mini-batches, the approach proves effective and stable
- one takeaway here is that, when applying batch normalization, the choice of batch size may be even more significant than without batch normalization

3.9 Batch normalization

- formally, denoting by $\mathbf{x} \in \mathcal{B}$ an input to batch normalization (BN) that is from a mini-batch $\mathcal{B}$, batch normalization transforms $\mathbf{x}$ according to the following expression:

$$\text{BN}(\mathbf{x}) = \gamma \odot \frac{\mathbf{x} - \hat{\mu}_{\mathcal{B}}}{\hat{\sigma}_{\mathcal{B}}} + \beta. \tag{4}$$

- in (4), $\hat{\mu}_{\mathcal{B}}$ is the sample mean and $\hat{\sigma}_{\mathcal{B}}$ is the sample standard deviation of the mini-batch $\mathcal{B}$
- after applying standardization, the resulting mini-batch has zero mean and unit variance
- because the choice of unit variance (vs. some other number) is an arbitrary choice, we commonly include element-wise *scale parameter* γ and *shift parameter* β, which have the same shape as $\mathbf{x}$
- note that γ and β are parameters that need to be learned jointly with the other model parameters

3.9 Batch normalization

- consequently, the variable magnitudes for intermediate layers cannot diverge during training, because batch normalization actively centers and rescales them back to a given mean and size (via $\hat{\mu}_{\mathcal{B}}$ and $\hat{\sigma}_{\mathcal{B}}$)
- one piece of practitioner's intuition or wisdom is that batch normalization seems to allow for more aggressive learning rates
- formally, we calculate $\hat{\mu}_{\mathcal{B}}$ and $\hat{\sigma}_{\mathcal{B}}$ in (4) as follows:

$$\begin{aligned} \hat{\mu}_{\mathcal{B}} &= \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x} \in \mathcal{B}} \mathbf{x} \\ \hat{\sigma}^2_{\mathcal{B}} &= \frac{1}{|\mathcal{B}|} \sum_{\mathbf{x} \in \mathcal{B}} (\mathbf{x} - \hat{\mu}_{\mathcal{B}})^2 + \epsilon. \end{aligned}$$

- note that we add a small constant $\epsilon > 0$ to the variance estimate to ensure that we never attempt division by zero, even in cases where the empirical variance estimate might vanish
- the estimates $\hat{\mu}_{\mathcal{B}}$ and $\hat{\sigma}_{\mathcal{B}}$ counteract the scaling issue by using noisy estimates of mean and variance
- we might think that this noisiness should be a problem; as it turns out, this is actually beneficial

3.9 Batch normalization

- fixing a trained model, we might think that we would prefer using the entire dataset to estimate the mean and variance
- once training is complete, why would we want the same image to be classified differently, depending on the batch in which it happens to reside?
- during training, such exact calculation is infeasible, because the intermediate variables for all data examples change every time we update our model
- however, once the model is trained, we can calculate the means and variances of each layer's variables based on the entire dataset
- indeed, this is standard practice for models employing batch normalization, and thus batch normalization layers function differently in *training mode* (normalizing by mini-batch statistics) and in *prediction mode* (normalizing by dataset statistics)

3.9 Batch normalization

- we are now ready to take a look at how batch normalization works in practice
- batch normalization implementations for fully-connected layers and convolutional layers are slightly different; we discuss both cases
- recall that one key difference between batch normalization and other layers is that, because batch normalization operates on a full mini-batch at a time, we cannot just ignore the batch dimension, as we did before, when introducing other layers

3.9 Batch normalization

- when applying batch normalization to fully-connected layers, the original paper inserts batch normalization after the affine transformation and before the nonlinear activation function (later applications may insert batch normalization right after activation functions)
- denoting the input to the fully-connected layer by $\mathbf{x}$, the affine transformation by $\mathbf{Wx} + \mathbf{b}$ (with the weight parameter $\mathbf{W}$ and the bias parameter $\mathbf{b}$), and the activation function by ϕ, we can express the computation of a batch-normalization-enabled, fully-connected layer output $\mathbf{h}$ as follows:

$$\mathbf{h} = \phi(\mathrm{BN}(\mathbf{Wx} + \mathbf{b})).$$

- recall that mean and variance are computed on the *same* mini-batch on which the transformation is applied

3.9 Batch normalization

- similarly, with convolutional layers, we can apply batch normalization after the convolution and before the nonlinear activation function
- when the convolution has multiple output channels, we need to carry out batch normalization for *each* of the outputs of these channels, and each channel has its own scale and shift parameters, both of which are scalars
- assume that our mini-batches contain m examples, and that, for each channel, the output of the convolution has height p and width q
- for convolutional layers, we carry out each batch normalization over the $m \cdot p \cdot q$ elements per output channel, simultaneously
- thus, we collect the values over all spatial locations when computing the mean and variance, and consequently apply the same mean and variance within a given channel to normalize the value at each spatial location

3.9 Batch normalization

- as we mentioned earlier, batch normalization typically behaves differently in training mode and prediction mode
- first, the noise in the sample mean and the sample variance arising from estimating each on mini-batches are no longer desirable, once we have trained the model
- second, we might not have the luxury of computing per-batch normalization statistics
- for example, we might need to apply our model to make one prediction at a time
- typically, after training, we use the entire dataset to compute stable estimates of the variable statistics, and then fix them at prediction time
- consequently, batch normalization behaves differently during training and at test time; recall that dropout also exhibits this characteristic